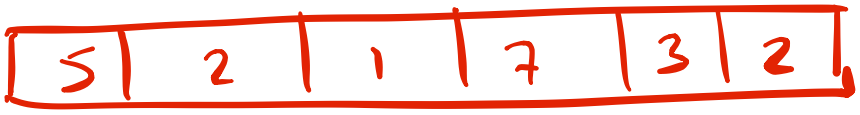
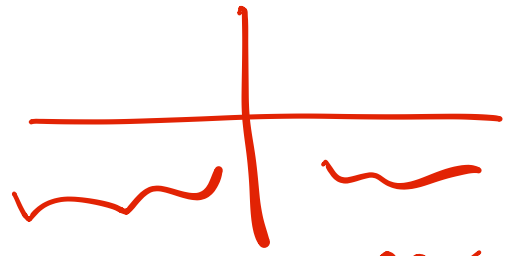
5.1

1. **a.** Write pseudocode for a divide-and-conquer algorithm for finding the position of the largest element in an array of n numbers.
   b. What will be your algorithm's output for arrays with several elements of the largest value?
   c. Set up and solve a recurrence relation for the number of key comparisons made by your algorithm.
   d. How does this algorithm compare with the brute-force algorithm for this problem?



a)



Algorithm FindLargest(A, l, r)

if l == r return l // one element

else

ind1 ← FindLargest(A, l, ⌊(l+r)/2⌋)

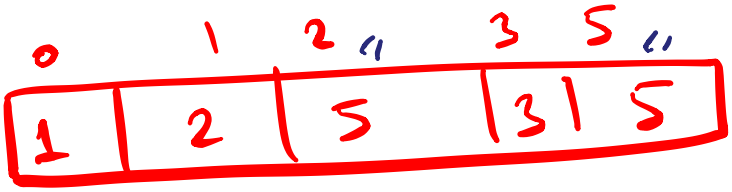
ind2 ← FindLargest(A, ⌊(l+r)/2⌋+1, r)

if A[ind1] ≥ A[ind2]

return ind1

else return ind2

b)



- return leftmost largest elem index → 2

c) $c(n) = 2c(n/2) + 1$, for $n > 1$

$c(1) = 0$

$n = 2^k$

$c(2^k) = 2\,c(2^{k-1}) + 1$

$c(2^{k-1}) = 2 \cdot c(2^{k-2}) + 1$

$= 2 \cdot [2 \cdot c(2^{k-2}) + 1] + 1$

$= 2^2 \cdot c(2^{k-2}) + 2 + 1$

$c(2^{k-2}) = 2 \cdot c(2^{k-3}) + 1$

$$= 2^2\left[2 \cdot c\left(2^{k-3}\right) + 1\right] + 2 + 1$$

$$= 2^3 \cdot c\left(2^{k-3}\right) + 2^2 + 2 + 1$$

$$\vdots$$

$$= 2^i \cdot c\left(2^{k-i}\right) + 2^{i-1} + 2^{i-2} + \cdots + 2 + 1$$

$$= 2^k \cdot c\left(2^{k-k}\right) + 2^{k-1} + 2^{k-2} + \cdots + 2 + 1$$

$$= \underbrace{2^{k-1} + 2^{k-2} + \cdots + 1}$$

$$\frac{2^{k-1+1} - 1}{2 - 1} = 2^k - 1$$

$$c(n) = n - 1 \rightarrow O(n)$$

d) Algorithm FindLargest-BF (A)

```
max ← -∞
for i = 0 to n-1
    if A[i] > max
        max ← A[i]
return max
```

$\Theta(n)$

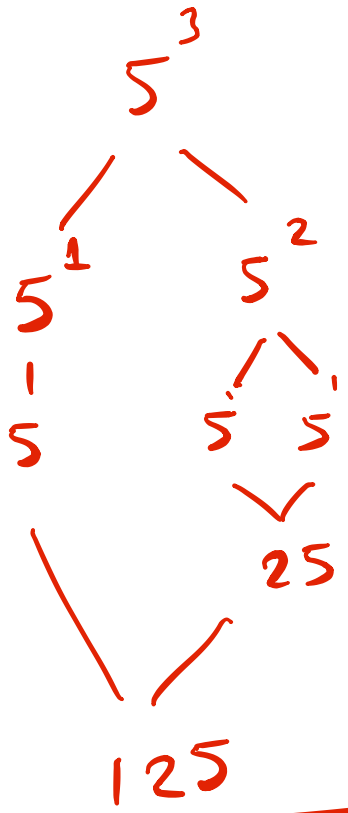
5.2

3. a. Write pseudocode for a divide-and-conquer algorithm for the exponentiation problem of computing a^n where n is a positive integer.

b. Set up and solve a recurrence relation for the number of multiplications made by this algorithm.

c. How does this algorithm compare with the brute-force algorithm for this problem?

a) 5^3



$$a^n = a^{\lfloor n/2 \rfloor} \cdot a^{\lceil n/2 \rceil}$$

Algorithm Power-DC (a, n)

if n == 1 return a

else return Power-DC (a, ⌊n/2⌋) * Power-DC (a, ⌈n/2⌉)

5) $C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + 1$ for $n > 1$

↓ # of multiplications

$C(1) = 0$

Use $n = 2^k$

$$C(2^k) = \underbrace{C(2^{k-1}) + C(2^{k-1})} + 1$$

$$= 2 \cdot C(2^{k-1}) + 1$$

$C(2^{k-1}) = 2 \cdot C(2^{k-2}) + 1$

$$= 2[2 \cdot C(2^{k-2}) + 1] + 1$$

$$= 2^2 \cdot C(2^{k-2}) + 2 + 1$$

$C(2^{k-2}) = 2 \cdot C(2^{k-3}) + 1$

$$= 2^2[2 \cdot C(2^{k-3}) + 1] + 2 + 1$$

$$= 2^3 \cdot C(2^{k-3}) + 2^2 + 2 + 1$$

$$= 2^i \cdot C(2^{k-i}) + 2^{i-1} + 2^{i-2} + \cdots + 1$$

$$= 2^k \cdot c(2^{k-k}) + 2^{k-1} + 2^{k-2} + \cdots + 1$$

$$= 2^{k-1} + 2^{k-2} + \cdots + 1$$

$$\frac{2^{k-1+1} - 1}{2-1} = 2^k - 1$$

$$c(n) = n - 1$$

c) Design Brute Force Alg. to compute a^n

Algorithm Power-BF (a , n)

res ← 1

for i = 0 to n-1 →

res ← res * a

return res

→ Time Complexity ?

$$\sum_{i=0}^{n-1} 1 = n$$

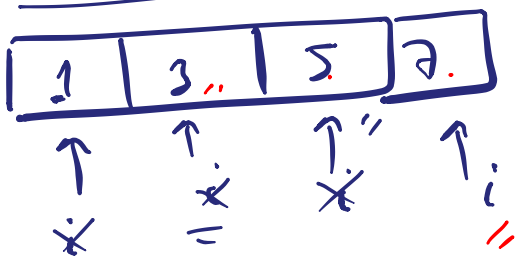
5.2

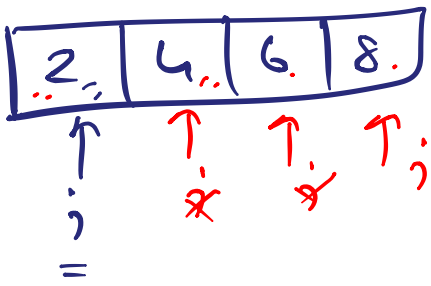
8. a. Solve the recurrence relation for the number of key comparisons made by mergesort in the worst case. You may assume that $n = 2^k$.

b. Set up a recurrence relation for the number of key comparisons made by mergesort on best-case inputs and solve it for $n = 2^k$.

a)

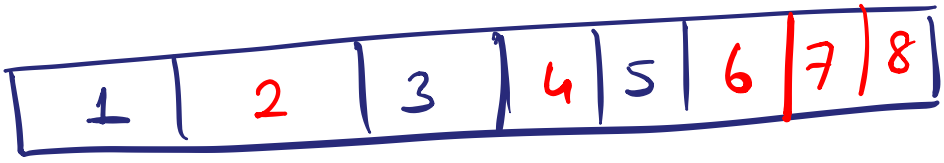
worstcase





n = 8

(n-1)



$C_w(n) = 2C_w(n/2) + n - 1$ merge for $n > 1$

$C_w(1) = 0$

$\rightarrow n = 2^k$

$C_w(2^k) = 2 \cdot C_w(2^{k-1}) + 2^k - 1$

$C_w(2^{k-1}) = 2 \cdot C_w(2^{k-2}) + 2^{k-1} - 1$

$= 2[2 \cdot C_w(2^{k-2}) + 2^{k-1} - 1] + 2^k - 1$

$= 2^2 \cdot C_w(2^{k-2}) + 2^k - 2 + 2^k - 1$

$C\omega(2^{k-2}) =$

$= 2^2 \left[2 \cdot C\omega(2^{k-3}) + 2^{k-2} - 1 \right]$ $\quad 2 \cdot C\omega(2^{k-3}) + 2^{k-2} - 1$

$= 2^3 \cdot C\omega(2^{k-3}) + 2^k - 2^2 - 2 - 1$

$= 2^3 \cdot C\omega(2^{k-3}) + 3 \cdot 2^k - (2^2 + 2 + 1)$

$\vdots$

$= 2^i C\omega(2^{k-i}) + i \cdot 2^k - (2^{i-1} + 2^{i-2} + \cdots + 1)$

$= 2^k C\omega(2^{k-k}) + k \cdot 2^k - (2^{k-1} + 2^{k-2} + \cdots + 1)$

$C\omega(2^0) = 0$

$= k \cdot 2^k - \dfrac{(2^{k-1} + 2^{k-2} + \cdots + 1)}{2^{k-1+1} - 1}$

$\dfrac{2^{k-1+1} - 1}{2 - 1}$

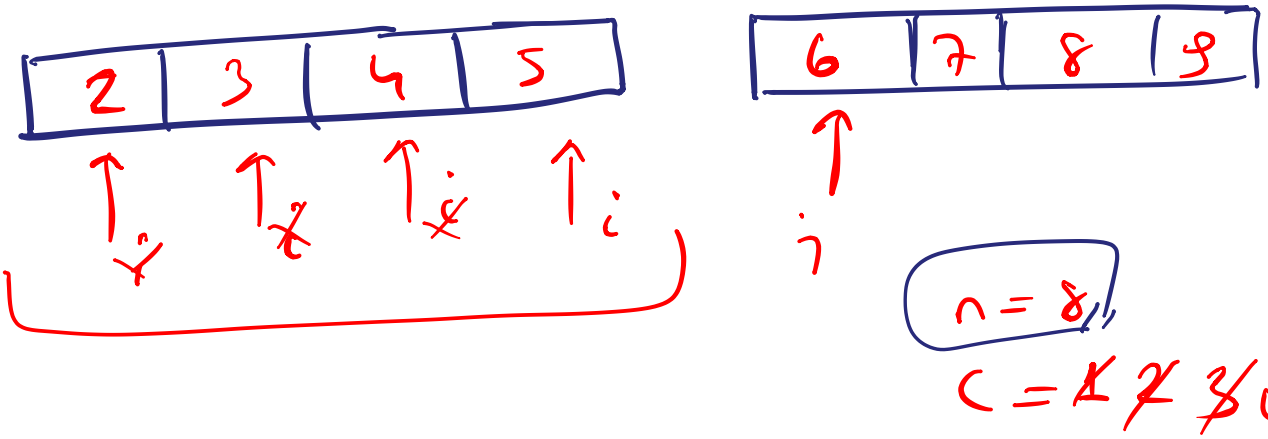
$= k \cdot 2^k - 2^k + 1$

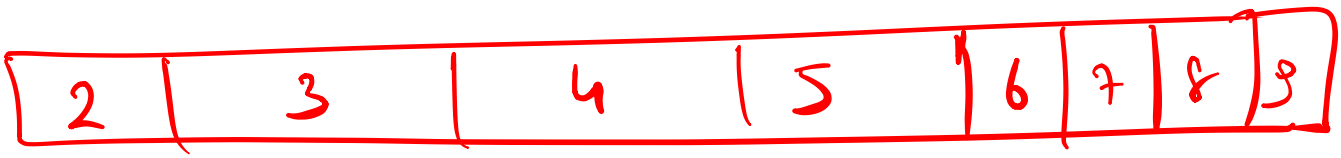
$= 2^k \cdot (k-1) + 1 = n \cdot (\lg n - 1) + 1$

$= n \lg n - n + 1$

b)

Best case





$$C_b(n) = 2C_b(n/2) + n/2 \quad \text{for } n>1$$

$$C_b(1) = 0$$

$$n = 2^k$$

$$C_b(2^k) = 2 \cdot C_b(2^{k-1}) + 2^{k-1}$$

$$\hookrightarrow C_b(2^{k-1}) = 2 \cdot C_b(2^{k-2}) + 2^{k-2}$$

$$= 2 \cdot \left[2 \cdot C_b(2^{k-2}) + 2^{k-2}\right] + 2^{k-1}$$

$$= 2^2 \cdot C_5(2^{k-2}) + \underbrace{2 \cdot 2^{k-2}}_{2^{k-1}} + 2^{k-1}$$

$$2^{k-1} + 2^{k-1} = 2 \cdot 2^{k-1}$$

$$= 2^2 \cdot C_5(2^{k-2}) + 2 \cdot 2^{k-1}$$

$$C_5(2^{k-2}) = 2 \cdot C_5(2^{k-3}) + 2^{k-3}$$

$$= 2^2 \left[2 C_5(2^{k-3}) + 2^{k-3} \right] + 2 \cdot 2^{k-1}$$

$$= 2^3 \cdot C_5(2^{k-3}) + \underbrace{2^2 \cdot 2^{k-3}}_{2^{k-1}} + 2 \cdot 2^{k-1}$$

$$2^{k-1} + 2^{k-1} = 3 \cdot 2^{k-1}$$

$$= 2^3 \cdot C_5(2^{k-3}) + 3 \cdot 2^{k-1}$$

$$\vdots$$

$$= 2^i \cdot C_5(2^{k-i}) + i \cdot 2^{k-1}$$

$$= 2^k \cdot \underbrace{C_5(2^{k-k})}_{0} + k \cdot 2^{k-1}$$

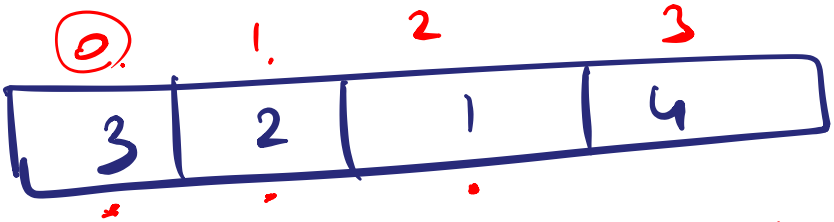
$$= k \cdot 2^{k-1} \Rightarrow \log n \cdot \frac{2^k}{2} = \frac{1 \log n \cdot n}{2}$$

5.1

9. Let $A[0..n-1]$ be an array of n real numbers. A pair $(A[i], A[j])$ is said to be an ***inversion*** if these numbers are out of order, i.e., $i < j$ but $A[i] > A[j]$. Design an $O(n \log n)$ algorithm for counting the number of inversions.

Reverse pair (A)

$i < j$ $\quad A[i] > A[j]$



For 3, $3 > 2 \rightarrow \checkmark$ +1
$3 > 1 \rightarrow \checkmark$ +1
$3 \not> 4 \rightarrow \times$
] 2

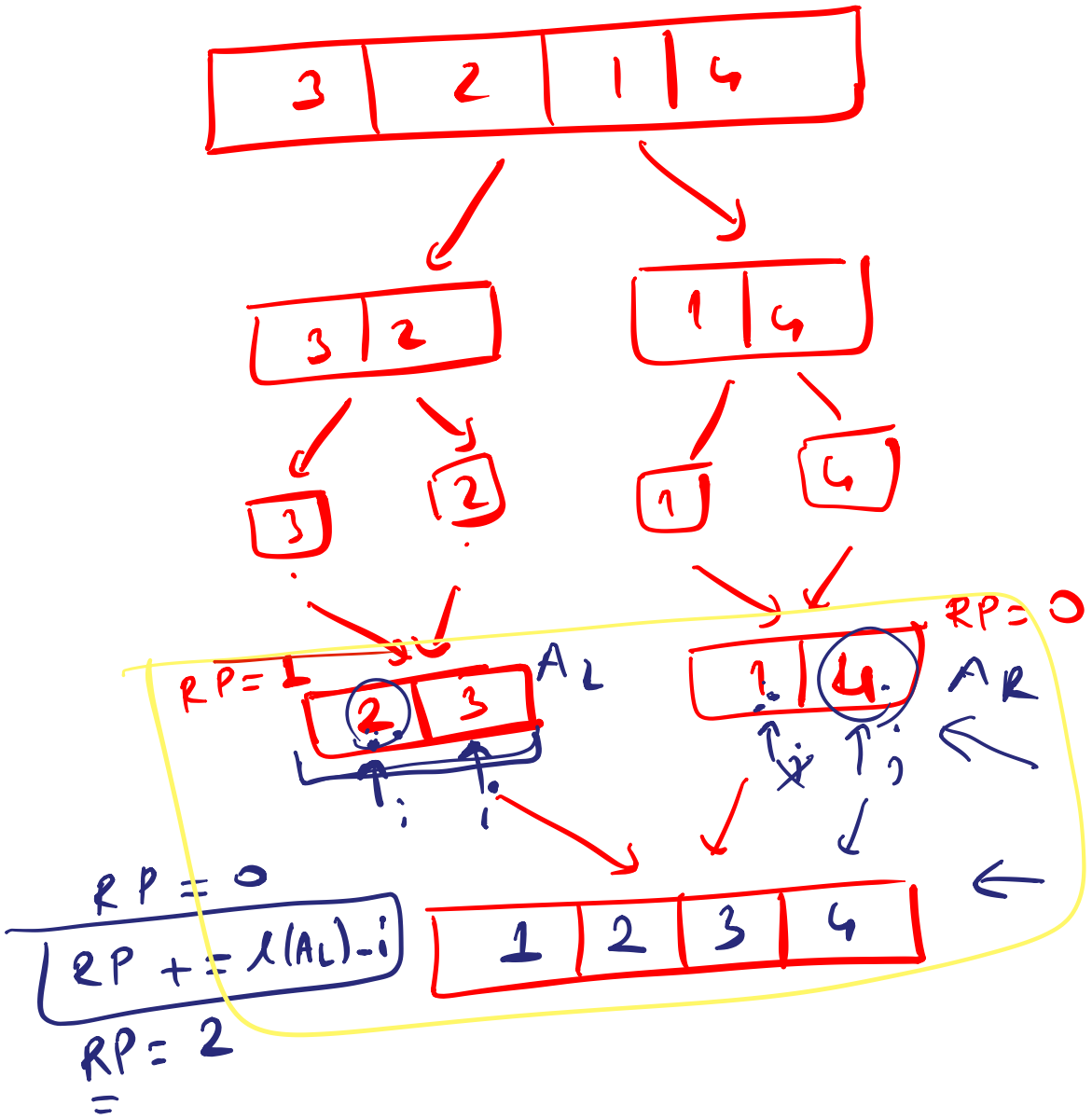
For 2, $2 > 1 \checkmark$
$2 \not> 4 \times$
] 1

For 1, $1 \not> 4 \times$

$\rightarrow$ 3

Algorithm ReversePair_BF(A)
c ← 0
for i = 0 to n − 2
  for j = i+1 to n−1
    if A[i] > A[j] → c++
return c

n^2



$$Tot = 1 + 0 + 2 = 3$$

Algorithm Sort_Count (A) → RP

if size(A) == 1 return (0, A)

else

$RP_L, A_L \leftarrow$ Sort_Count (A[0, ..., mid-1])

$RP_R, A_R \leftarrow$ Sort_Count (A[mid, ..., n-1])

$RP_m, A_{sorted} \leftarrow$ Merge_Count (A_L, A_R)

Alg. mergeCount (A_L, A_R)

L : size of A_L

R : size of A_R

Asorted ← ∅

i ← 0

j ← 0

count ← 0

while (i < L & j < R)

→ if A[i] > A[j]

count += L - i

Append A[j] to Asorted

j++

else

Append A[i] to Asorted

i++

Append all remaining elements to the Asorted

return count, Asorted

$$T(n) = 2\,T(n/2) + O(n)$$

5.2

8. Design an algorithm to rearrange elements of a given array of n real numbers so that all its negative elements precede all its positive elements. Your algorithm should be both time efficient and space efficient.

Algorithm NegPos(A)

$i \leftarrow 0$, $j \leftarrow n-1$

while $i \leq j$ do

if $A[i] < 0$

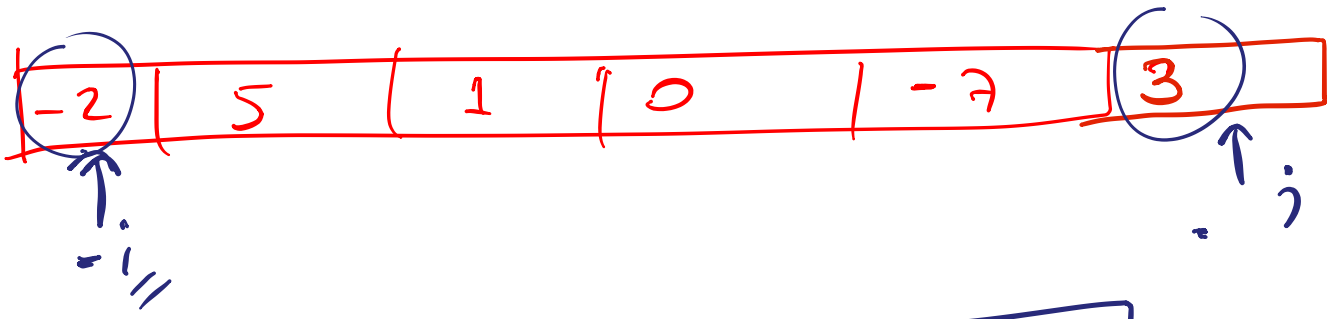
$i \leftarrow i+1$

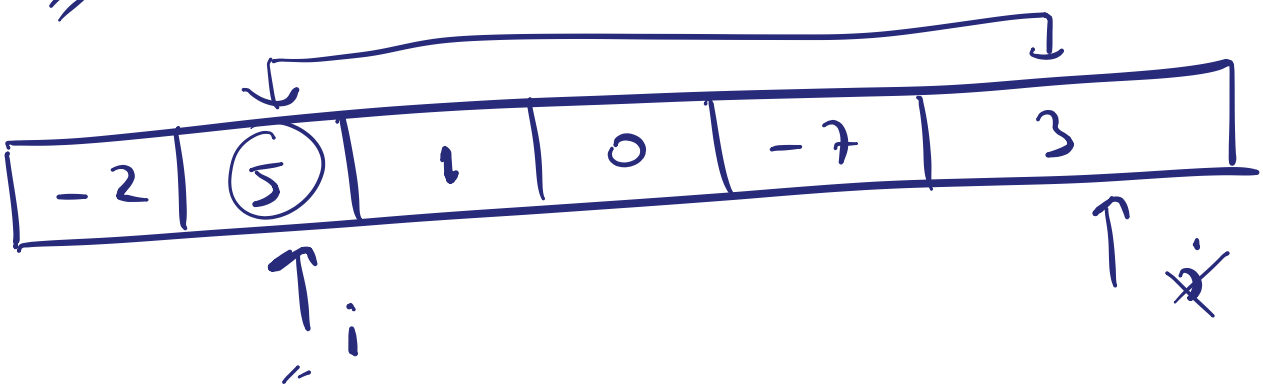
else

swap $(A[i], A[j])$

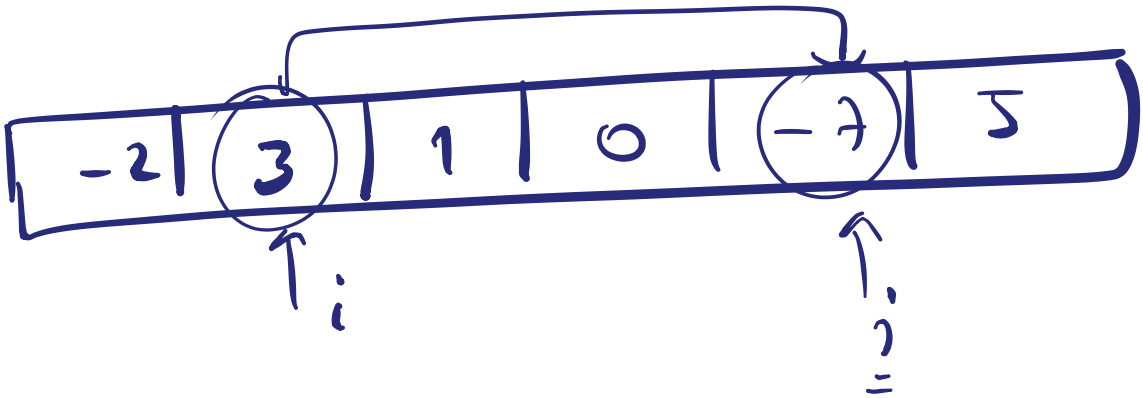
$j--$

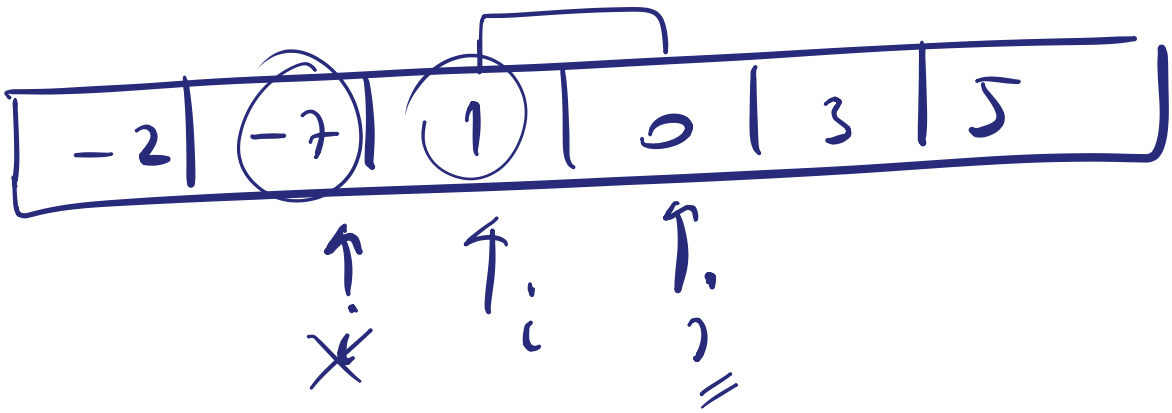


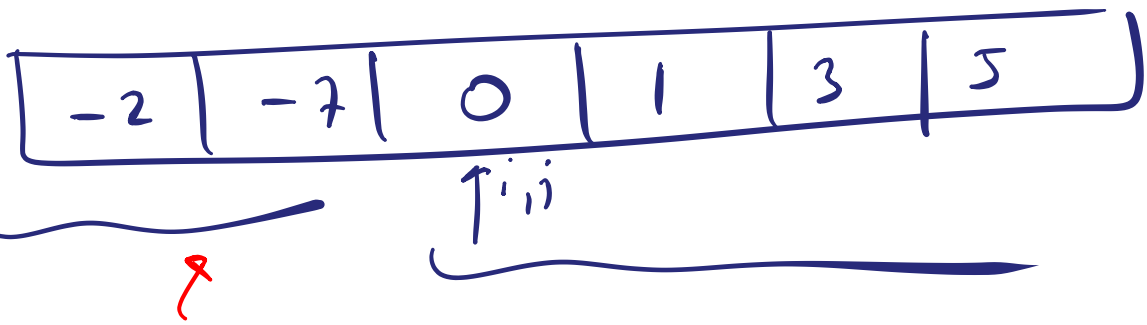




→ A[i] < 0







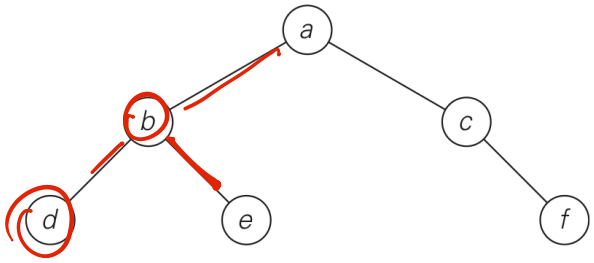
5.3

5. Traverse the following binary tree

a. in preorder. N L R

b. in inorder. L N R

c. in postorder. L R N



a) a b d e c f

b) d b e a c f

c) d e b f c a

5.5

1. a. For the one-dimensional version of the closest-pair problem, i.e., for the problem of finding two closest numbers among a given set of n real numbers, design an algorithm that is directly based on the divide-and-conquer technique and determine its efficiency class.

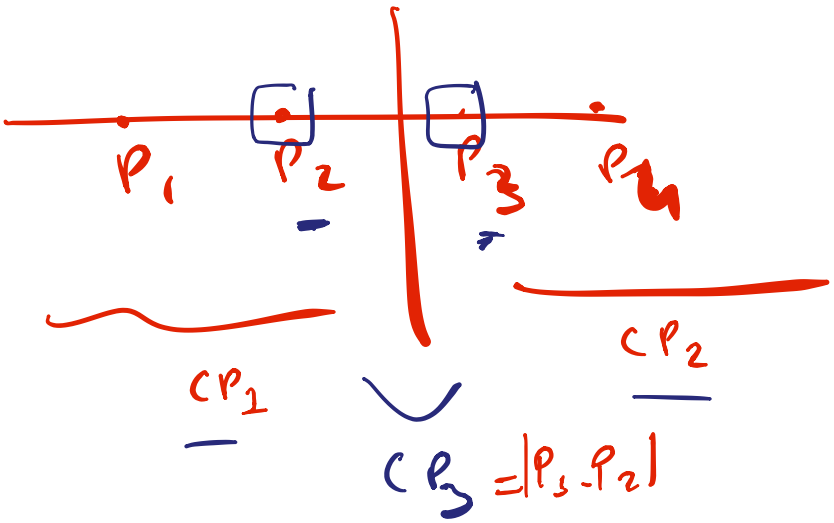


$A = [P_2, P_1, \ldots, P_n]$

→ $\Theta(n \log n)$

→ Sort the data points in increasing order

→



$\min \{ CP_1, CP_2, CP_3 \}$

Algorithm CP_DC(P_S ↗ sorted)

if $l == r$ return ∞

else if $l - r == 1$ return $P[r] - P[l]$

else

$CP_1 \leftarrow CP_DC(P_s[0, \ldots, \lfloor \frac{l+r}{2} \rfloor])$

$CP_2 \leftarrow CP_DC(P_s[\lfloor \frac{l+r}{2} \rfloor + 1, \ldots, r])$

$CP_3 \leftarrow P_s[\lfloor \frac{l+r}{2} \rfloor + 1] - P[\lfloor \frac{l+r}{2} \rfloor]$

- Determine min $\{CP_1, CP_2, CP_3\}$

return closest pairs

$$T(n) = 2T(n/2) + 1 \quad \text{for } n > 2$$

$T(1) = 0$

$T(2) = 1$

$T(n) \in \Theta(n)$

Total Time complexity $= \Theta(n \log n) + \Theta(n)$

$\Theta(n \log n)$ ← sorting

$= \Theta(n \log n)$